MINI PROJECT REPORT

BITCOIN PRICE PREDICTION

With Investment Recommendation

Project Details:

- **Topic:** Bitcoin Price Prediction Web App
- **Frontend:** HTML, CSS
- **Backend:** Python, Flask
- **Machine Learning:** Linear Regression

1. Project Overview

Currently, many people are investing in Bitcoin without proper or basic knowledge. If they use our website, they can easily find the approximate future or past value for a Bitcoin on any random date. This website helps users understand the price trend and gives them advice on whether to buy or sell.

This project is a clean, simple web application that predicts Bitcoin prices for the next 7 days based on 1 year of historical data. The application lets a user pick a specific date, uses a Machine Learning model (Linear Regression) to calculate the price, and displays a recommendation badge (BUY, SELL, or HOLD) alongside a trend graph.

2. Project Structure (Files Used)

The project is made of simple files organized in a folder structure:

- **index.html:** The website interface file where the user selects dates and looks at results.
- **prediction.py:** The backend Python script running Flask that handles page logic, fetches live data, trains the model, and gives predictions.
- **bitcoin_price_model.pkl:** The saved trained machine learning file.

3. Frontend Design (HTML & CSS)

The frontend is designed to be user-friendly, responsive, and clean. It uses a modern light-blue gradient background to look beautiful and neat.

- **Input Form:** A clean calendar dropdown box allows users to easily pick a date and hit a green "Submit" button.
- **Result Section:** Shows the predicted target price clearly in Indian Rupees (INR) alongside a bold recommendation message.
- **Action Status:** Uses colored button cards (Green for BUY, Red for SELL) to give clear visual guidance.

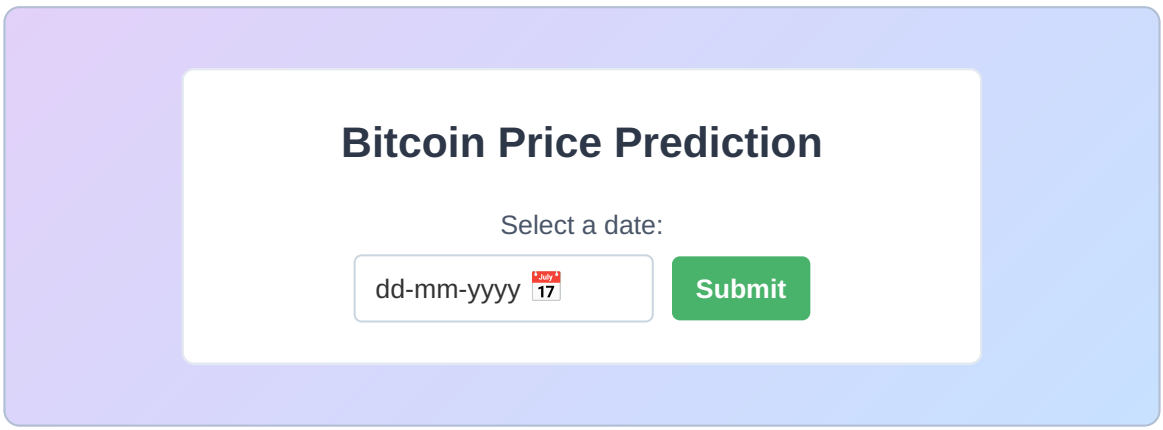

Figure 1: Clean Web Interface Home Page Layout

4. Backend Logic & Machine Learning

The backend logic is built with the Python Flask framework. When the application runs, it follows these clear steps:

1. **Data Collection:** It fetches historical daily Bitcoin closing prices for the past 365 days from the live CoinGecko API.
2. **Data Preprocessing:** The system splits each date into numerical parts (Day, Month, and Year) so the model can read it easily.
3. **Model Training:** A Linear Regression model is trained using these Day, Month, and Year inputs to fit the price history.
4. **7-Day Prediction:** When a user submits a date, the app forecasts the prices for the next 7 days straight.

```
# Simple snippet of backend model calculation
from sklearn.linear_model import LinearRegression

# Train model on historical data
model = LinearRegression()
model.fit(X_train, y_train)

# Predict next 7 days prices
predicted_prices = model.predict(future_dates)
```

5. Investment Logic Engine

The application decides what action to recommend by looking at the price difference between Day 1 and Day 7 of the predicted timeline:

- **BUY:** If the predicted prices are going up over the next 7 days, a green **BUY** button is displayed.

- **SELL:** If the predicted prices are going down, a red **SELL** button is shown to warn the user.
- **HOLD:** If the price remains stable, it recommends holding current assets.

6. System Output Screenshots & Case Studies

Below are two neat examples of how the app handles predictions depending on whether the trend is down or up:

Case 1: Negative Price Trend (SELL Example)

When the user picks 26/04/2024, the application predicts a downward trend. It recommends selling holdings to avoid losses.

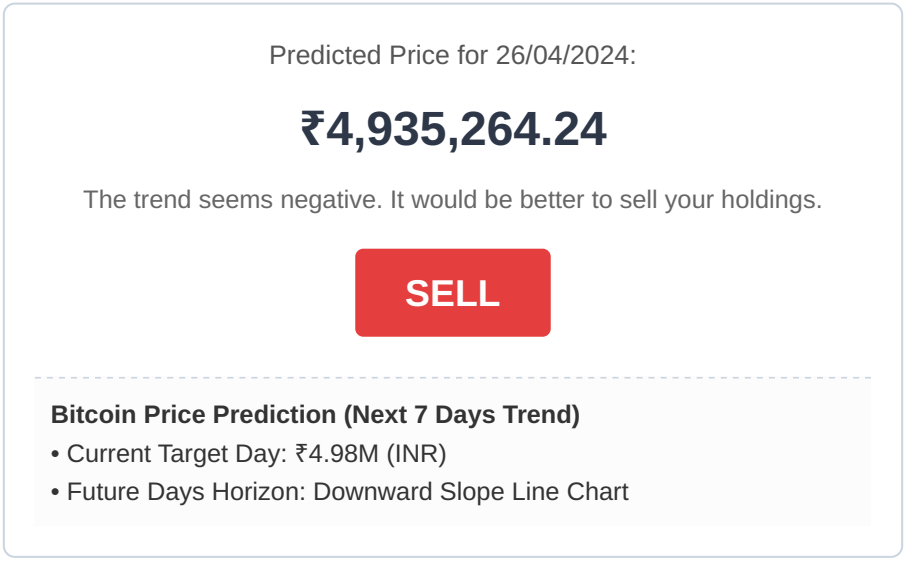

Figure 2: Output UI screen showing Negative Trend and SELL advice

Case 2: Positive Price Trend (BUY Example)

When the user inputs a future date like 09/11/2027, the line trends upward. The app displays a green buy signal.

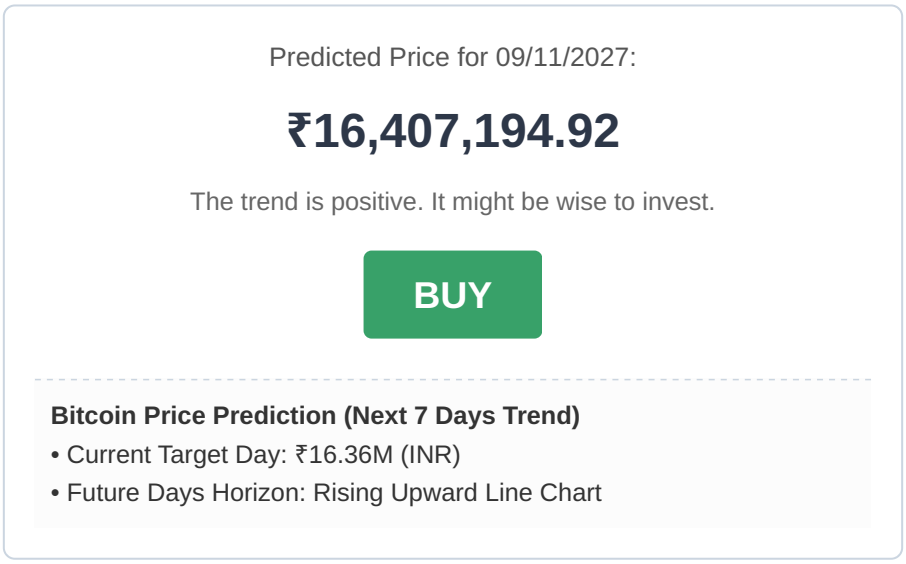

Figure 3: Output UI screen showing Positive Trend and BUY advice

7. Summary Table of App Files

File Name	Role in Project	Main Function
index.html	Frontend View	Displays the calendar input and handles results UI.
prediction.py	Backend Code	Fetches data from API, runs Machine Learning model, and outputs recommendations.
bitcoin_price_model.pkl	Saved Model Data	Stores the trained weights for instant prediction lookups.

8. Conclusion

This mini-project provides a clean, simple solution to help normal users analyze Bitcoin trends before making investment decisions. By combining HTML/CSS design with a Flask backend and a basic Linear Regression model, the web app effectively turns hard numbers into simple, clean instructions like **BUY** or **SELL** with an easy-to-read layout.